# Relational & Logical Operators in C

# 1.4 Relational Operator

- Relational operators are used to compare two variables.
- It will return 1 , if the comparison or relation is true.
- otherwise it will return 0.
- They determine the relation between the two data.
- The result of the operation will be either **TRUE (1)** or **FALSE (0)**.

| OPERATOR | MEANING | EXAMPLE | RESULT |
|---|---|---|---|
| < | Less than | 1<2 | True |
| > | Greater than | 1>2 | False |
| <= | Less than or equal to | 1<=2 | True |
| >= | Greater than or equal to | 1>=2 | False |
| == | Equal to | 1==2 | False |
| != | Not equal to | 1!=2 | True |

# Contd...

| Operator | Description | Example (true=1,false=0) |
|---|---|---|
| == | Checks both left and right operands are same | 10 == 10 it will return true.<br>10 == 7 it will return false |
| != | Checks both left and right operands are not same. | 10 != 20 it will return true.<br>10 != 10 it will return false. |
| < | Checks whether the left operand is smaller than the right operand. | 5 < 10 it will return true.<br>10 < 10 it will return false. |
| <= | Checks whether the left operand is smaller than or equal to the right operand. | 10 <= 10 it will return true<br>20 <= 10 it will return false. |
| > | Checks whether the left operand is greater than the right operand. | 50 > 10 it will return true<br>5 > 10 it will return false. |
| >= | Checks whether the left operand is greater than or equal to the right operand. | 10 >= 10 it will return true<br>5 >= 10 it will return false. |

# Sample Program

```c
#include<stdio.h>

int main()
{
    int a = 5, b = 10;

    printf("Return value of (%d == %d) is %d\n", a, b, a == b); //return 0
    printf("Return value of (%d != %d) is %d\n", a, b, a != b); //return 1
    printf("Return value of (%d <  %d) is %d\n", a, b, a < b);  //return 1
    printf("Return value of (%d <= %d) is %d\n", a, b, a <= b); //return 1
    printf("Return value of (%d >  %d) is %d\n", a, b, a > b);  //return 0
    printf("Return value of (%d >= %d) is %d\n", a, b, a >= b); //return 0

    return 0;
}
```

# 1.5 Logical Operator

- Logical operators are used to perform logical operations of given expressions (relational expressions) or variables.
- There are three logical operators available in C.

**| | (Logical OR) operator**

**&& (Logical AND) operator**

**! (Logical NOT) operator**

- The operands as well as the result of these operators are Boolean values (**TRUE** and/or **FALSE**).
- When we want to test more than one condition and make decisions, **&&** and **| |** operators are used and hence they are **binary operators**.
- But **logical NOT ( ! )** is a **unary operator** as works with **one relational operation**.

# || (Logical OR) Operator

- If one of the operands or expressions is true, it will return 1.
- If all of them are false, it will return 0.

| A | B | A \|\| B | Example |
|---|---|---|---|
| 0 | 0 | 0 | (5 > 10) \|\| (5 < 4) Both expressions are false. so, logical OR output will be 0 |
| 0 | 1 | 1 | (10 > 20) \|\| (10 < 20) First expression is false and second one is true. so, logical OR output will be 1 |
| 1 | 0 | 1 | (10 < 20) \|\| (10 > 100) First expression is true and second one is false. so, logical OR output will be 1 |
| 1 | 1 | 1 | (10 < 20) \|\| (10 < 100) Both expressions are true. so, logical OR output will be 1 |

# && (Logical AND) Operator

- If both left and right operands or expressions are true, it will return true. Otherwise, it will return false.
- Note, non-zero value operands are considered as true.

| A | B | A && B | Example |
|---|---|---|---|
| 0 | 0 | 0 | (5 > 10) && (5 < 4) Both expressions are false. so, logical AND output will be 0 |
| 0 | 1 | 0 | (10 > 20) && (10 < 20) First expression is false and second one is true. so, logical AND output will be 0 |
| 1 | 0 | 0 | (10 < 20) && (10 > 100) First expression is true and second one is false. so, logical AND output will be 0 |
| 1 | 1 | 1 | (10 < 20) && (10 < 100) Both expressions are true. so, logical AND output will be 1 |

# ! (Logical NOT) Operator

- Logical NOT operator is used to inverse the current decision.
- Say, if current state is true, Logical NOT (!) operator will make it false.

| A | !A | Example |
|---|---|---|
| 0 | 1 | !(100 < 10) 100 is greater than 10. So, it will return false.<br>!(false) ==> true |
| 1 | 0 | !(10 < 100) 10 is less than 100. So, it will return true.<br>!(true) ==> false |

# Sample Program

```
#include<stdio.h>
int main()
{
    int a = 5, b = 10 ,ret;

    ret = ((a <= b) || (a != b));
    printf("Return value of above expression is %d\n",ret);

     ret = (( a < b) && (a == b ));
    printf("Return value of above expression is %d\n",ret);

    ret = ! (( a < b) && (a == b ));
    printf("Return value of above expression is %d\n",ret);
    return 0;
 }
```

# Sample Program

```c
#include<stdio.h>
int main()
{
    int a = 5, b = 10 ,ret;

    ret = ( (a <= b) || (a != b) );
    // 5 <= 10 ==> true. 5 != 10 ==> true. So, 1 || 1 will return 1.
    printf("Return value of above expression is %d\n",ret);

    ret = ( ( a < b) && (a == b ) );
    // 5 < 10 ==>true. 5 == 10 ==> false. So, 1 && 0 will return 0;
    printf("Return value of above expression is %d\n",ret);

    ret = ! ( ( a < b) && (a == b ) );
    /*we have used the same expression here.
     And its result was 0.
     So !0 will be 1.*/
    printf("Return value of above expression is %d\n",ret);

    return 0;
 }
```

# Control Statements in C

# 2 Control Statements

- Control statements are used to decide the program execution order based on some conditions.
- It is also called **decision making statements** or **branching statements**.
- It provides decision making capability to a program.
- There are **two conditional statements** in C: **if** and **switch**.
- The **if statement** has many versions such as **simple if, if-else, if-else-if.**

- C has the following conditional statements:

  Use **if** to specify a block of code to be executed, if a specified condition is true

  Use **else** to specify a block of code to be executed, if the same condition is false

  Use **else if** to specify a new condition to test, if the first condition is false

  Use **switch** to specify many alternative blocks of code to be executed

# Contd…

## Example

- Let’s write a program for dividing two integers.
- But, if we divide a number by 0, program execution will stop as its undefined behavior.
- Here, we must decide and restrict the execution of dividing code, if the divisor is zero.
- Dividing a number by zero is undefined.
- So, the execution order will be,

```
If divisor is zero
    Don’t execute division statement.
Else
    Execute it.
```

# 2.1 If Statement

- Statements under if executes only, if the given condition is true.
- Otherwise those statements will not be part of program.

**Syntax:**

```
if(condition)
{
    //statements
}
```

```
if(0) //always false
{
    //statements will not execute
}
```

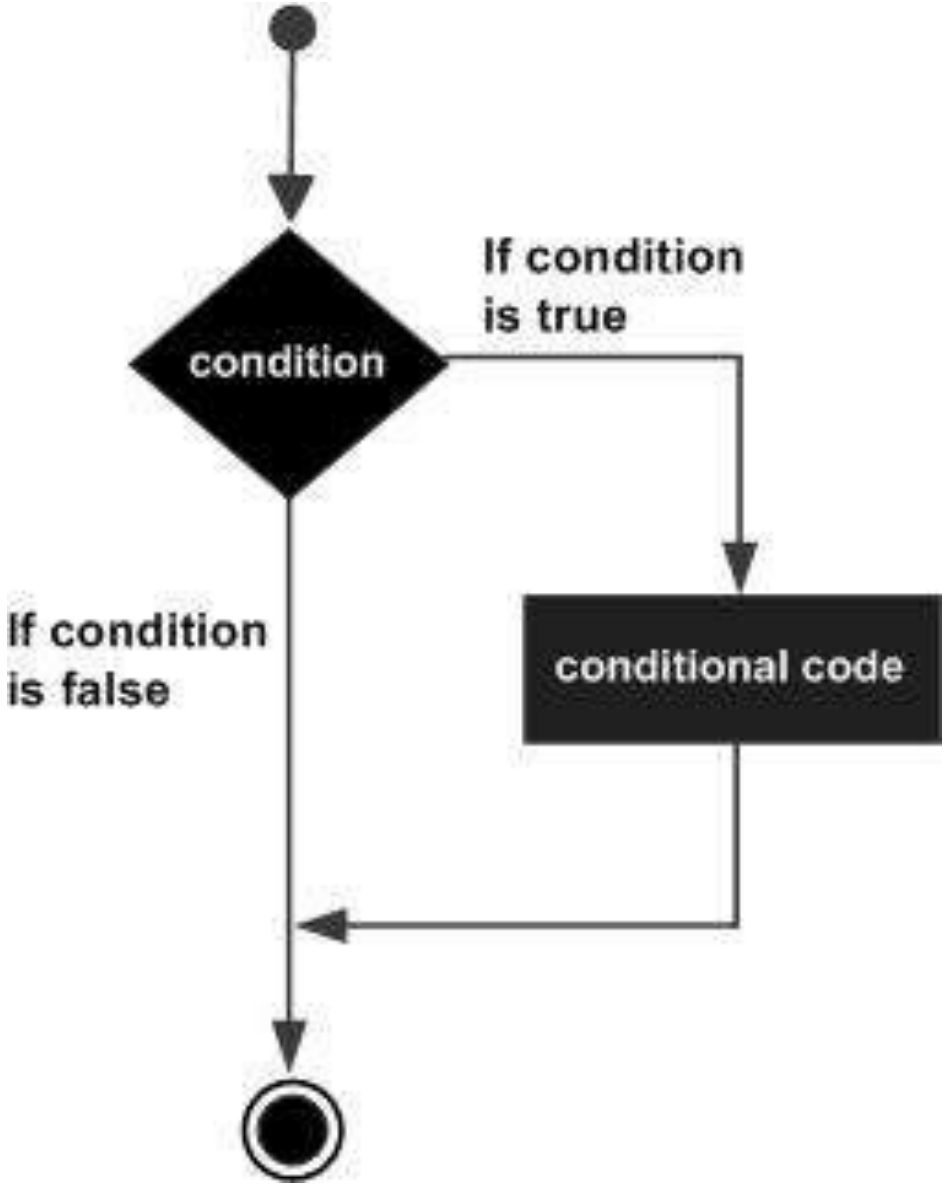
```
if(non-zero) //always true
{
    //statements will execute
}
```

- Likewise, when we give some expression or logical combination of expression in if statement, the statements under `if` only work, if the expressions output is non-zero.
- Note that `if` is in lowercase letters. Uppercase letters (**`If`** or **`IF`**) will generate an error.

## Syntax of if statement

```
if(condition)
{
    //statement 1
    //statement 2
    //statement n
}
```

## Flowchart of if statement



**Program:** Dividing two numbers. We should restrict the execution, if the divisor value is 0.

```
#include<stdio.h>
int main()
{
    int num1,num2;
    printf("Enter two numbers for division");
    scanf("%d%d",&num1,&num2);

    // if num2 value not equal to zero, it will become if(1).
    // otherwise it will become if(0)

    if(num2 != 0)
    {
        printf("num1 / num2 = %d\n",num1/num2);
    }
    return 0;
}
```

# WAP in C to print the absolute value of a number

```
#include<stdio.h>
int main()
{
  int num;
  printf(“Enter a integer number”);
  scanf(“%d”, &num);
  if(num<0)
  {
     num = -num;
  }
  printf("The number is:%d\n",num);

  return 0;
}
```

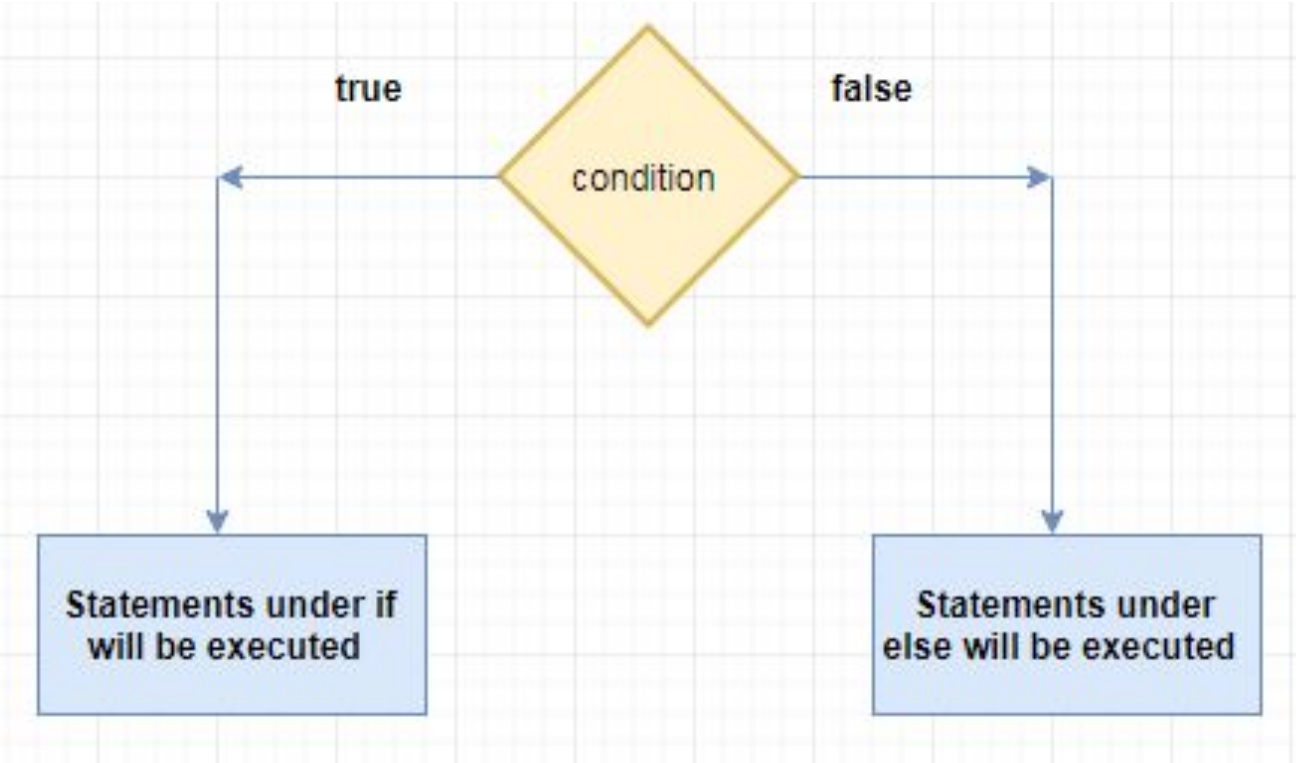
# 2.2 If else Statement

- Statements under `if` executes only, when the given condition is true.
- Otherwise, the statements under `else` will be executed.
- When we need to decide either this or that in programming, we should go for if-else statement.

  Like, **positive or negative**, **prime or composite**, **odd or even** etc.

**Syntax:**

```
if(condition)
{
    //statements
}
else
{
    //statements
}
```



# Sample Program

- Get a number from user and say whether it is zero or non-zero number.

```
#include<stdio.h>
int main()
{
    int num;
    scanf("%d",&num);

    if(num == 0)
    {
        printf("Zero");
    }
    else
    {
        printf("Non-zero");
    }

    return 0;
}
```